SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CSA16 **COURSE CODE:** Data Warehousing and Data Mining

---

COURSE OUTCOME COVERED

CO3: Evaluate the effectiveness of classification techniques on real-time data for accurate prediction, enabling informed decision-making. (BL3, BL4, BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 3 **Total Marks: 30**

Annexure C

Case study

Criteria	Understanding of Case (2.5)	Application of Concepts (2.5)	Analysis (2)	Solution Design (2)	Clarity (1)	Total (10)
Weightage	25%	25%	20%	20%	10%	**100%**
Q1						
Q2						
Q3						

Code of Conduct:

I **Kongara Jogesh (192525035)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kongara Jogesh

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Annexure A

Case Study – Solutions

Question 1 (10 Marks)

The story of the shepherd boy who cried "Wolf!" is a classic analogy for a binary classifier's confusion matrix. Here, the boy's cry is the model's 'prediction' (Wolf present / Wolf absent) and the true situation in the field is the 'actual' class.

Confusion Matrix Framework:

	Predicted: Wolf (Positive)	**Predicted: No Wolf (Negative)**
Actual: Wolf (Positive)	TP	FN
Actual: No Wolf (Negative)	FP	TN

Mapping the story to TP, FP, FN, TN:

Event	**Villagers' Belief (Prediction)**	**Reality (Actual)**	**Classification**
1st & 2nd cry (boy cries wolf, none present)	Wolf	No Wolf	False Positive (FP)
3rd time: real wolf comes, villagers stay home	No Wolf	Wolf	False Negative (FN)
A night with no wolf & no cry (hypothetical)	No Wolf	No Wolf	True Negative (TN)
If villagers had believed the 3rd cry (hypothetical)	Wolf	Wolf	True Positive (TP)

• False Positive (FP): The boy cried "Wolf!" for fun when no wolf was present. The model (boy) predicted 'Wolf' but the actual class was 'No Wolf' — a false alarm.

• False Negative (FN): When the real wolf came and the boy cried "Wolf!", the villagers (acting as the decision system) predicted 'No Wolf' because they no longer trusted the alarm, while the actual class was 'Wolf' — a missed detection with serious consequences (the flock was lost).

• True Positive (TP): Would have occurred if the villagers had believed the boy's warning during the real wolf attack and responded correctly — this did NOT happen in the story, which is exactly the tragedy being illustrated.

• True Negative (TN): Would occur on any night when there was no wolf and the boy correctly did not cry "Wolf!" — not explicitly narrated, but implied for the many uneventful nights before the story begins.

Key Insight / Lesson:

This story illustrates the real-world danger of a high False Positive rate: repeated false alarms erode trust in a warning/classification system, causing the system's later correct alarms (when the actual danger appears) to be ignored — resulting in a False Negative with severe consequences. In machine learning and alert systems, this motivates the need to balance Precision (avoiding false alarms) with Recall (not missing true events), since either extreme carries a real cost.

Question 2 (10 Marks)

There are 100 images: 30 contain a cat (actual positive) and 70 do not (actual negative). The model correctly predicts 'cat' for 25 of the 30 cat images, and correctly predicts 'no cat' for 50 of the 70 no-cat images.

Step 1: Identify TP, FP, FN, TN

• True Positive (TP) = 25 — cat images correctly predicted as cat

• False Negative (FN) = 30 − 25 = 5 — cat images incorrectly predicted as no cat

• True Negative (TN) = 50 — no-cat images correctly predicted as no cat

• False Positive (FP) = 70 − 50 = 20 — no-cat images incorrectly predicted as cat

Confusion Matrix:

	Predicted: Cat	**Predicted: No Cat**	**Total**
Actual: Cat (30)	TP = 25	FN = 5	30
Actual: No Cat (70)	FP = 20	TN = 50	70
Total	45	55	100

Step 2: Precision, Recall, F1 Score

Metric	**Formula**	**Calculation**	**Value**
Precision	TP / (TP + FP)	25 / (25 + 20) = 25/45	0.556 (55.6%)
Recall (Sensitivity)	TP / (TP + FN)	25 / (25 + 5) = 25/30	0.833 (83.3%)
F1 Score	2 × (P × R) / (P + R)	2 × (0.556 × 0.833) / (0.556 + 0.833)	0.667 (66.7%)
Accuracy	(TP + TN) / Total	(25 + 50) / 100	0.750 (75.0%)

Interpretation:

• Precision (55.6%) — of all images the model labelled as 'cat', only about 56% actually contained a cat; the model produces a fair number of false alarms (20 FP).

• Recall (83.3%) — of all the actual cat images, the model successfully found about 83% of them, missing only 5.

• F1 Score (66.7%) — the harmonic mean of precision and recall shows a moderate overall balance, pulled down by the relatively low precision.

• The model is better at finding cats (high recall) than it is at being correct when it claims an image is a cat (lower precision) — it should be tuned to reduce false positives if precision matters more for the use case.

Question 3 (10 Marks)

Sample Dataset:

#	Offer	Free	Length	Class
1	Yes	Yes	Short	Spam
2	Yes	No	Long	Not Spam
3	No	Yes	Short	Spam
4	No	No	Long	Not Spam
5	Yes	Yes	Long	Spam
6	No	No	Short	Not Spam

1. Prior Probabilities

Out of 6 emails: 3 are Spam, 3 are Not Spam.

P(Spam) = 3/6 = 0.5

P(Not Spam) = 3/6 = 0.5

2. Conditional Probabilities for Each Feature

Given Class = Spam (rows 1, 3, 5):

Feature | Class = Spam	Value	Conditional Probability
Offer = Yes	2 of 3	2/3 = 0.667
Offer = No	1 of 3	1/3 = 0.333
Free = Yes	3 of 3	3/3 = 1.000
Free = No	0 of 3	0/3 = 0.000
Length = Short	2 of 3	2/3 = 0.667
Length = Long	1 of 3	1/3 = 0.333

Given Class = Not Spam (rows 2, 4, 6):

Feature | Class = Not Spam	Value	Conditional Probability
Offer = Yes	1 of 3	1/3 = 0.333
Offer = No	2 of 3	2/3 = 0.667
Free = Yes	0 of 3	0/3 = 0.000
Free = No	3 of 3	3/3 = 1.000

Length = Short	1 of 3	1/3 = 0.333
Length = Long	2 of 3	2/3 = 0.667

3. Classify (Offer = Yes, Free = No, Length = Short) using Naïve Bayes

Applying the Naïve Bayes formula:

P(Spam | X) *P(Spam) × P(Offer=Yes|Spam) × P(Free=No|Spam) × P(Length=Short|Spam)*
= 0.5 × 0.667 × 0.000 × 0.667 = 0.000

P(NotSpam | X) *P(NotSpam) × P(Offer=Yes|NotSpam) × P(Free=No|NotSpam) × P(Length=Short|NotSpam)*
= 0.5 × 0.333 × 1.000 × 0.333 = 0.0556

Problem — Zero Frequency: Since 'Free = No' never appears among the Spam training examples, P(Free=No | Spam) = 0, which forces the entire Spam score to 0 regardless of the other features. This is the classic zero-frequency problem in Naïve Bayes, and it is normally solved using Laplace (add-one) smoothing.

Applying Laplace Smoothing (add-1, 2 categories per feature):

Feature	P(. | Spam) smoothed	P(. | Not Spam) smoothed
Offer = Yes	(2+1)/(3+2) = 0.600	(1+1)/(3+2) = 0.400
Free = No	(0+1)/(3+2) = 0.200	(3+1)/(3+2) = 0.800
Length = Short	(2+1)/(3+2) = 0.600	(1+1)/(3+2) = 0.400

Smoothed Spam score = 0.5 × 0.600 × 0.200 × 0.600 = 0.036

Smoothed Not-Spam score = 0.5 × 0.400 × 0.800 × 0.400 = 0.064

Normalised: P(Spam) = 0.036/0.100 = 36%, P(Not Spam) = 0.064/0.100 = 64%

Conclusion: The email (Offer=Yes, Free=No, Length=Short) is classified as NOT SPAM, since 0.064 > 0.036 (equivalently, the Not-Spam class has the higher posterior probability, 64% vs 36%).

4. Comparing Results Using WEKA (Naïve Bayes Classifier)

Loading this dataset into WEKA's Explorer and running the 'NaiveBayes' classifier (Classify tab) uses Laplace/kernel-estimator smoothing by default, so it avoids the zero-probability issue seen in the raw manual calculation. Running WEKA on this dataset (6 instances, 3 nominal attributes, Naïve Bayes with default settings) is expected to produce the same overall decision — the instance (Offer=Yes, Free=No, Length=Short) is classified as 'Not Spam' — because WEKA's internal Laplace correction matches the smoothed calculation performed above. WEKA's output panel would additionally report the predicted class probabilities (close to ~0.36 Spam / ~0.64 Not Spam) and a summary confusion matrix / cross-validation accuracy for the training set, letting us confirm the manual computation is correct.

5. Discussion: The Independence Assumption in Naïve Bayes

Naïve Bayes assumes that all features are conditionally independent of one another given the class label — i.e., knowing that an email 'Contains Offer' gives no additional information about whether it also 'Contains Free', once we already know the class (Spam / Not Spam).

• In reality, this assumption is often violated: spam emails that contain the word "Offer" very frequently also contain the word "Free", so these two features are actually correlated rather than independent.

• Despite this 'naïve' and technically incorrect assumption, the algorithm tends to work well in practice because it only needs the ranking of posterior probabilities across classes to be correct — not their exact magnitude — for classification decisions to be accurate.

• The independence assumption is what makes Naïve Bayes extremely fast, simple, and effective on small datasets like this one (only 6 training records), since it needs to estimate only a few simple conditional probabilities per feature rather than a full joint distribution.

• Its main weakness is the zero-frequency problem shown above (fixed via Laplace smoothing) and reduced accuracy when features are strongly correlated, since double-counting correlated evidence can bias the posterior probability of the class that shares those correlated features.